Boucles et répétitions (Python)

Robotique – Grade 10 – Plan de cours de 2 heures

Enseignant : Francis Ki | Semaine du 16 au 20 novembre 2026

1 Informations générales

Titre	Boucles `for` et `while`
Classe / effectif	Grade 10, 11 à 20 élèves, niveaux hétérogènes, élèves à besoins particuliers
Durée	2 heures (120 min)
Lieu	Salle informatique, un poste par élève
Matériel et logiciels	— Un poste par élève avec l'éditeur **Mu** en mode *Python 3* (bouton Arrêter pour interrompre une boucle infinie, Débogueur pour suivre l'exécution). — Vidéoprojecteur ou tableau interactif avec Mu projeté. — Un dossier partagé contenant les fichiers de départ `.py` des trois niveaux. — Inspiration : unité « Round and round » du cours TeachComputing (binômes, codage en direct, exemples commentés, idées fausses courantes).
Pré-requis	Séances précédentes : variables, types, saisie, conditions `if / elif / else` et indentation.

Préparation avant le cours (environ 40 min)

1. Préparer les fichiers de départ : `niveauA.py` (boucle for à compléter), `niveauB.py` (vide avec consignes), `niveauC.py` (début du jeu « devine le nombre »).
2. Tester le bouton Arrêter de Mu avec une boucle infinie volontaire.
3. Créer le formulaire de sortie (annexe G) ou imprimer l'annexe.

Prévenir la boucle infinie

Montrer dès la mini-leçon ce qui se passe avec `while True` sans arrêt, et comment arrêter l'exécution dans Mu. Cela évite la panique en classe et sert de leçon sur l'erreur de logique.

2 Objectifs d'apprentissage

Objectifs du programme

— Utiliser les boucles `for` et `while`.

— Automatiser des traitements répétitifs.

À la fin de la séance, l'élève est capable :

— d'écrire une boucle `for` avec `range` et de prévoir les valeurs parcourues ;

— d'écrire une boucle `while` avec une condition d'arrêt et un compteur ;

— de choisir entre `for` (nombre de tours connu) et `while` (jusqu'à une condition).

Compétences numériques : suivre l'exécution avec une table de trace, utiliser le débogueur, interrompre un programme.

3 Déroulé minuté (120 min)

Minutes	Étape	Contenu	Organisation
0 – 10	Accroche	« Je ne bavarderai plus » cent fois	Classe entière
10 – 30	Mini-leçon	`for`, `range`, `while`	Classe entière
30 – 60	Pratique guidée	Une table de multiplication en collectif, puis un mot de passe en binômes	Collectif puis binômes
60 – 95	Pratique autonome	Trois niveaux de besoin	Individuel sur poste
95 – 110	Mise en commun	Valoriser chaque groupe	Classe entière
110 – 120	Exit ticket	Trois questions sur poste	Individuel sur poste

1. Accroche : cent lignes de punition — 0 – 10 min

1. (5 min) *Un enseignant demande d'écrire cent fois « Je ne bavarderai plus ». Comment un ordinateur le ferait-il sans que vous écriviez cent lignes ?* Les élèves proposent des idées.
2. (5 min) Écrire deux versions au tableau : cent `print` recopiés, ou une ligne qui répète. Question-problème : **« Comment dire à l'ordinateur de répéter une action ? »**

2. Mini-leçon : for et while — 10 – 30 min

Support : annexe A. Codage en direct. Quatre temps de 5 min.
Temps 1 : for et range.

Boucle for

```
for i in range(5):
    print("Tour numero", i)
```

`range(5)` donne 0, 1, 2, 3, 4 (cinq valeurs, on commence à 0). `range(1, 6)` donne 1 à 5. `range(0, 10, 2)` donne 0, 2, 4, 6, 8.
Temps 2 : while.

Boucle while

```
n = 3
while n > 0:
    print(n)
    n = n - 1
print("Decollage !")
```

La boucle se répète *tant que* la condition est vraie. Il faut modifier la variable pour que la condition devienne fausse.
Temps 3 : la table de trace.

Tour	Valeur de `n` avant	Affichage
1	3	3
2	2	2
3	1	1

Temps 4 : choisir. `for` quand on connaît le nombre de tours, `while` quand on répète jusqu'à ce qu'une condition change (mot de passe, saisie valide, jeu).

Différenciation et besoins particuliers

Faire suivre la table de trace avec un doigt ou un stylo. Utiliser le débogueur de Mu pour voir la variable changer ligne après ligne. Rappeler le piège classique : `range(5)` ne donne pas 5.

3. Pratique guidée — 30 – 60 min

Fiche : annexe B. Programmation en binômes (pilote et navigateur, rôles échangés à mi-temps).
3a. Table de multiplication en collectif (12 min). Écrire en codage en direct un programme qui demande un nombre puis affiche sa table de 1 à 10 avec une boucle `for`.
3b. Mot de passe en binômes (18 min). Écrire un programme qui redemande un mot de passe tant que la réponse est fausse (`while`), puis affiche « Acces autorise ».
Étayage progressif de l'aide.

Niveau d'aide	Contenu	Qui intervient
Aide 1	Le programme du 3a et l'annexe A avec la table de trace	Le binôme seul
Aide 2	Questions guides : *Combien de fois la boucle se répète-t-elle ? Quelle variable change à chaque tour ? Quand la condition devient-elle fausse ?*	Le binôme avec la carte d'aide
Aide 3	L'enseignant écrit la ligne `while` avec le binôme, qui continue	L'enseignant

Différenciation et besoins particuliers

Écrire d'abord le programme en français (« tant que le mot de passe est faux, redemander »), puis le traduire en Python. Si la boucle ne s'arrête pas, cliquer sur Arrêter et chercher la ligne qui devrait modifier la variable.

4. Pratique autonome différenciée — 60 – 95 min

Consigne commune : chaque élève ouvre le fichier de son groupe, le renomme avec son prénom et travaille seul sur son poste. L'enseignant passe d'abord auprès du groupe A pendant 10 min.

Groupe	Tâche sur poste	Annexe
A : en dessous du niveau	Fichier `niveauA.py` à compléter : boucle `for` qui affiche les nombres de 1 à 10, puis table de trace à remplir, avec une banque de mots (`for`, `in`, `range`, `print`).	Annexe C
B : au niveau	Écrire trois programmes : table de multiplication choisie par l'utilisateur, somme des nombres de 1 à n avec `for`, mot de passe à trois essais avec `while`. Tester chacun avec deux jeux d'essais.	Annexe D
C : au-dessus du niveau	Réaliser le jeu « devine le nombre » : l'ordinateur choisit un nombre entre 1 et 100 (`random.randint`), le joueur propose, le programme indique « plus grand » ou « plus petit », compte les essais et s'arrête quand le nombre est trouvé. Ajouter une limite de 7 essais.	Annexe E

Plan B si un élève termine tôt : afficher un triangle d'étoiles avec deux boucles imbriquées.

5. Mise en commun — 95 – 110 min

1. (2 min) Retour au calme, écrans mis en veille.
2. (9 min) Trois passages courts de 3 min, avec valorisation :
   - **Groupe A** : un élève présente sa boucle `for` et sa table de trace. Valoriser la prédiction avant l'exécution.
   - **Groupe B** : un binôme montre son mot de passe à trois essais et explique la condition d'arrêt. Valoriser la précision.
   - **Groupe C** : un élève fait jouer la classe à « devine le nombre ». Valoriser la structure et l'interactivité.
3. (4 min) Conclusion : *une boucle automatise une répétition : `for` pour un nombre de tours connu, `while` tant qu'une condition est vraie.*

6. Clôture : exit ticket — 110 – 120 min

Formulaire en ligne ou papier (annexe G) :

1. Quelles valeurs prend `i` dans `for i in range(4):` ?
2. Pourquoi la boucle `while n > 0:` ne s'arrête-t-elle jamais si on oublie `n = n - 1` ?
3. Auto-évaluation : *Je maîtrise / Je progresse / J'ai besoin d'aide.*

Lire les réponses avant la séance suivante pour ajuster les groupes de besoin.

Annexe A : Support résumé de la mini-leçon

Boucles en Python

Nom : _____ Date : _____

for : nombre de tours connu. **while** : tant que la condition est vraie. Dans les deux cas, le bloc répété est **indenté**.

for

```
for i in range(1, 6):
    print(i)
```

while

```
essais = 0
while essais < 3:
    print("Essai", essais + 1)
    essais = essais + 1
```

Instruction	Valeurs
`range(5)`	0, 1, 2, 3, 4
`range(1, 6)`	1, 2, 3, 4, 5
`range(0, 10, 2)`	0, 2, 4, 6, 8

Annexe B : Fiche de pratique guidée

Pratique guidée : répéter Nom : _____ Date : _____

3a. Table de multiplication

À compléter pendant le codage en direct

```
n = int(input("Quelle table ? "))
for i in range(1, _____):
    print(n, "x", i, "=", n * _____)
```

3b. Mot de passe

À compléter

```
mot = input("Mot de passe : ")
while mot != "robot":
    mot = _____("Mot de passe : ")
print("Acces autorise")
```

Tour	Valeur de `mot`	La condition est vraie ?
1		
2		
3		

Cartes d'aide à utiliser dans l'ordre

Aide 1 : regarde le programme du 3a et la table de trace de l'annexe A.
Aide 2 : Combien de fois la boucle se répète-t-elle ? Quelle variable change à chaque tour ?
Aide 3 : demande à l'enseignant d'écrire la ligne `while` avec toi.

Annexe C : Niveau A (en dessous du niveau)

Niveau A : je complète la boucle

Nom : _____ Date : _____

Banque de mots : for, in, range, print, 11, i.

niveauA.py

```
_____ i _____ range(1, _____):
    _____(i)
```

Activité : table de trace

Tour	Valeur de `i`
1	
2	
3	
10	

Activité : vrai ou faux ?

1. `range(5)` commence à 1. ☐ Vrai ☐ Faux
2. `range(1, 4)` donne 1, 2, 3. ☐ Vrai ☐ Faux

Annexe D : Niveau B (au niveau)

Niveau B : trois programmes Nom : _____ Date : _____

Programme 1 : table choisie

Demande un nombre et affiche sa table de 1 à 10.

Programme 2 : somme de 1 à n

Demande n et affiche la somme $1 + 2 + \cdots + n$. Indice : une variable `somme` qui part de 0.

Programme 3 : mot de passe à trois essais

Le programme s'arrête après un bon mot de passe ou après trois essais ratés.

Programme	Entrée testée	Résultat attendu	Obtenu
2	$n = 4$		
2	$n = 10$		
3	bon mot dès le 2e essai		
3	trois erreurs		

Annexe E : Niveau C (au-dessus du niveau)

Niveau C : devine le nombre Nom : _____ Date : _____

niveauC.py : début du programme

```
import random
secret = random.randint(1, 100)
essais = 0
trouve = False
while not trouve and essais < 7:
    # a toi de jouer : demander, comparer, compter
    pass
```

1. Complète la boucle : demande un nombre, indique « plus grand » ou « plus petit », compte les essais.
2. Affiche un message de fin différent en cas de victoire ou de défaite.
3. Combien d'essais suffisent toujours avec la stratégie de la dichotomie (couper l'intervalle en deux) ?

   Défi bonus : propose de rejouer à la fin avec une deuxième boucle.

Annexe F : Corrections et attendus (enseignant)

Annexe B

3a : `range(1, 11)`, `n * i`. 3b : `input`. La condition est vraie tant que le mot est différent de `"robot"`.

Annexe C

`for i in range(1, 11): print(i)`. Trace : 1, 2, 3, 10. Vrai ou faux : Faux (commence à 0) ; Vrai.

Annexe D

Somme : $n = 4$ donne 10 ; $n = 10$ donne 55. Mot de passe : `essais = 0` ; `while essais < 3 and mot != "robot"`. Cas limites : bon mot au 2e essai, ou trois erreurs.

Annexe E

Boucle attendue : `reponse = int(input(...))` ; `essais = essais + 1` ; `if reponse == secret: trouve = True` ; `elif reponse < secret: print("plus grand")` ; `else: print("plus petit")`. Dichotomie : 7 essais suffisent pour 1 à 100 ($2^7 = 128$).
Critères de réussite : boucle qui s'arrête, compteur correct, messages de fin distincts.

Annexe G : Exit ticket et grille d'observation

Exit ticket : boucles Nom : _____ Date : _____

1. Quelles valeurs prend `i` dans `for i in range(4):` ?

2. Pourquoi la boucle `while n > 0:` ne s'arrête-t-elle jamais si on oublie `n = n - 1` ?

3. Je me sens : ☐ Je maîtrise ☐ Je progresse ☐ J'ai besoin d'aide

Attendus : 1. 0, 1, 2, 3. 2. La condition reste toujours vraie, car `n` ne change pas.

Grille d'observation (enseignant)

Élève	`range` compris	Condition d'arrêt	Programme testé	Groupe suivant